Module (15) Advance Python Programming

(1) Printing on Screen

• Introduction to the print() function in Python.

- The print() function in Python is a built-in function used to display text, variables, or the results of calculations on the screen. It acts as a primary tool for communicating with users and is highly useful for debugging your code.

  Basic Syntax :

  print(“Hello World!”)

• Formatting outputs using f-strings and format().

- Python provides two modern, powerful ways to format text output, f-strings (Formatted String Literals) and the format() method. While f-strings are the preferred, faster, and more readable choice for most tasks, format() remains highly useful for delayed or reusable templates

(2) Reading Data from Keyboard

• Using the input() function to read user input from the keyboard.

- In Python, you read user input from the keyboard using the built-in input() function, which pauses program execution and waits for the user to type text and press the Enter key.

• Converting user input into different data types (e.g., int, float, etc.).

- In Python, the input() function always captures user data as a string (text) by default. If you want to perform math or logic on that input, you must manually transform it into a different data type through explicit type conversion, also known as type casting.

  Core Data Type Conversions

  (1) Integers (int)

  Use the Python int() function to convert a string of whole numbers.

  age = int(input("Enter your age: "))

  (2) Floats (float)

  Use the Python float() function to convert a string containing decimals or scientific notation.

price = float(input("Enter the item price: "))

(3) Booleans (bool)

Use the bool() function to evaluate truth values.

Note: Any non-empty string returns True, while an empty string returns False.

has_agreed = bool(input("Press Enter to clear, or type any key to agree: "))

(3) Opening and Closing Files

• Opening files in different modes ('r', 'w', 'a', 'r+', 'w+').

- The primary difference between file modes lies in whether they allow reading, writing, or both, whether they erase existing data, and how they handle missing files. Choosing the wrong mode can accidentally delete your data or crash your script.

  (1) Read Mode ('r')

  Behavior: Opens a file strictly for reading.

  Risk: If the file does not exist, it raises a FileNotFoundError.

  Use Case: Safely viewing file contents without risking accidental modifications.

  (2) Write Mode ('w')

  Behavior: Opens a file strictly for writing. It creates a new file if it's missing.

  Danger: If the file already exists, it completely deletes all existing text (truncates it) the moment you open it.

  Use Case: Creating brand-new documents or entirely replacing old logs.

  (3) Append Mode ('a')

  Behavior: Opens a file for adding new data. It creates the file if it doesn't exist.

  Safety: It leaves existing content untouched and forces the cursor to the very end of the file.

  Use Case: Continuously adding lines to error logs or chat histories.

  (4) Read and Write Mode ('r+')

  Behavior: Opens the file for both reading and writing, starting at the beginning.

  Nuance: It does not erase your file data on launch. However, writing to it will overwrite characters sequentially from the top.

Risk: Throws a FileNotFoundError if the target file is missing.

(5) Write and Read Mode ('w+')

Behavior: Opens a file for simultaneous writing and reading.

Danger: Just like regular 'w', it instantly wipes all file contents on opening. You can read back only what you write during that specific session.

• Using the open() function to create and access files.

- In Python, you can use the built-in open() function to create, read, and modify files by returning a file object that links your code to the storage system.

  The industry standard is to use open() inside a with statement, known as a context manager. This best practice guarantees that Python automatically closes the file after your code block executes, protecting against memory leaks and data corruption.

• Closing files using close().

- Closing files using the close() method terminates the connection between your program and a storage file, freeing up critical system resources. When you open a file, your operating system allocates a file descriptor (or handle) to track it. Because operating systems limit the total number of files a program can keep open simultaneously, you must explicitly close them when you are finished.

(4) Reading and Writing Files

• Reading from a file using read(), readline(), readlines().

- In Python, the read(), readline(), and readlines() methods allow you to retrieve data from an opened file. Each method handles memory differently and outputs data in a distinct format.

  (1) The read() Method

  The read() method extracts data from the file and returns it as a single string.

  Behavior: If no argument is passed, it reads the whole file. If a number n is provided, it reads exactly n characters (or bytes) from the current position of the file pointer.

  Syntax: file_object.read(size)

  (2) The readline() Method

  The readline() method reads one individual line from the file at a time, moving the file pointer to the next line for subsequent calls.

Behavior: It stops reading when it encounters a newline character (\n). It keeps the \n at the end of the string. It returns an empty string "" when the end of the file (EOF) is reached.

Syntax: file_object.readline(limit)

(3) The readlines() Method

The readlines() method processes the entire file and packs every line into a list of strings.

Behavior: Each list element represents one line from the text file, including its trailing \n character.

Syntax: file_object.readlines(hint)

• Writing to a file using write() and writelines().

- In Python, the write() method accepts a single string argument and writes it to a file, while writelines() accepts an iterable of strings (like a list) and writes multiple lines at once. Crucially, neither method adds newline characters (\n) automatically, meaning you must include them manually to format your text into separate lines.

  (1) Using the write() Method

  The write() method is designed for a single string. Passing a list directly into this method will trigger a TypeError.

  (2) Using the writelines() Method

  The writelines() method streamlines writing a collection of strings. It iterates through your sequence internally and pushes each element to the file.

(5) Exception Handling

• Introduction to exceptions and how to handle them using try, except, and finally.

- Exceptions are unexpected events that occur during program execution, causing the code to crash or interrupt. To prevent this, you use exception handling blocks to catch errors, gracefully handle them, and ensure critical cleanup tasks are always completed.

  Exception Handling

  try: The block of code where you test for potential errors. This is your "risky" code.

  except: The block of code that executes only if an error occurs in the try block. You can catch general exceptions or specific ones (like a ZeroDivisionError).

else: An optional block that runs *only* if the try block completes successfully without any errors.

finally: An optional block that *always* executes, regardless of whether an exception occurred or was handled. It is primarily used for cleanup tasks like closing files or database connections.

• Understanding multiple exceptions and custom exceptions.

- Multiple exceptions allow a program to catch and handle different types of errors generated by a single block of code, while custom exceptions are developer-defined error classes used to represent specific business logic failures that standard language exceptions cannot adequately describe.

  Example :

```
print("program started")
try :
    a = 10
    b = a/0
    print(b)
except ZeroDivisionError as e:
    print(e)
```

(6) Class and Object (OOP Concepts)

• Understanding the concepts of classes, objects, attributes, and methods in Python.

- In Python Object-Oriented Programming (OOP), classes serve as the architectural blueprints, objects are the physical instances built from those blueprints, attributes act as the data stored within them, and methods represent the executable actions they can perform.

  (1) Classes

  A class defines a brand-new data type. It encapsulates data structures and operational behaviors into a single organizational unit. It is written using the class keyword.

  (2) Objects

  An object is the tangible realization of a class. It occupies memory space and holds its own unique values for the properties outlined by the class template. You generate an object by calling the class name as if it were a function.

(3) Attributes

Attributes are variables bound to a class or an object instance. Python tracks them via dot-notation (object.attribute).

Instance Attributes: Unique to each single object. Defined inside the constructor method.

Class Attributes: Shared identically by every object instantiated from that class. Defined directly inside the main class body.

(4) Methods

Methods are functions embedded inside the class structure. Their first parameter must always be self, which passes a reference back to the specific object instance calling that method.

The __init__ Method: This is Python's constructor. It runs automatically whenever a new object is created to initialize its values.

• Difference between local and global variables.

- The primary difference between a local and global variable is their scope and lifetime within a program: local variables are declared inside a specific function or block and can only be accessed there, while global variables are declared outside of all functions and can be accessed from anywhere in the code.

  Example :

```
# global variable
a = 10
def test():
# local variable
    a = 20
    print(a)
test()
print(a)
```

(7) Inheritance

• Single, Multilevel, Multiple, Hierarchical, and Hybrid inheritance in Python.

- Python supports five distinct types of inheritance that structure relationships between base (parent) and derived (child) classes. Inheritance maximizes

code reuse by allowing a child class to automatically look up attributes and methods from its ancestral classes.

1 - Single Inheritance

A single child class inherits directly from one parent class. This is the simplest and most common form of inheritance.

2 - Multilevel Inheritance

A child class inherits from a parent class, which in turn inherits from another parent class. This establishes a linear grandparent-parent-child relationship.

3 - Multiple Inheritance

A single child class inherits attributes and methods from more than one parent class.

4 - Hierarchical Inheritance

Multiple child classes are derived from a single parent class. This is used when different classes share a common base configuration but require distinct individual actions.

5 - Hybrid Inheritance

Hybrid inheritance is a combination of two or more inheritance patterns outlined above (e.g., mixing multiple and hierarchical inheritance).

• Using the super() function to access properties of the parent class.

- The super() function in Python creates a temporary proxy object of the parent class, allowing you to seamlessly call its constructors, methods, and properties from a subclass. It is primarily used to eliminate code duplication and manage complex Method Resolution Order (MRO) in multiple inheritance setups.

  You will most frequently use super() inside the __init__ method of a child class to ensure that the parent class attributes are initialized correctly.

(8) Method Overloading and Overriding

• Method overloading: defining multiple methods with the same name but different parameters.

- Method overloading is a programming feature that allows a class to have multiple methods with the exact same name, provided their parameter lists are different. It is a core concept of Object-Oriented Programming (OOP) that implements compile-time (static) polymorphism. The compiler automatically

determines which specific method to execute based on the number, types, or order of the arguments passed into the function call.

Rules for Overloading Methods

To successfully overload a method in languages like Java, C#, or C++, the methods must differ in their method signature (the method name plus its parameters).

Number of parameters: Having a different count of inputs.

Data types of parameters: Having different types of inputs (e.g., int vs. double).

Sequence of parameters: Changing the order of data types (e.g., (String, int) vs. (int, String)).

Use Method Overloading For

Improves Readability: Avoids forcing developers to invent or remember bloated, redundant names like addInt(), addThreeInts(), and addDouble().

Code Consistency: Provides a unified interface for executing identical logical behavior across different types of data.

Flexibility: Cleanly manages optional arguments or alternative formats without crashing the program.

• Method overriding: redefining a parent class method in the child class.

- Method overriding occurs when a child class provides a specific implementation for a method that is already defined in its parent class. It allows a subclass to change or extend the behavior of inherited code to better suit its specific needs

  Key Rules

  Identical Signature: The child method must have the exact same name, parameter list, and return type as the parent method.

  Inheritance Required: Overriding can only happen if an "IS-A" relationship exists between the two classes.

  Runtime Resolution: The specific method execution is determined at runtime based on the actual object type, enabling runtime polymorphism.

  Restrictions: Static, final, and private methods cannot be overridden.

  Use For

  It enables a single interface to handle multiple distinct data types smoothly.

It allows you to leverage the existing attributes of a parent class while injecting customized logic.

It permits calling the original parent function inside the child class using the super keyword if you want to extend behavior rather than replace it entirely.

(9) SQLite3 and PyMySQL (Database Connectors)

• Introduction to SQLite3 and PyMySQL for database connectivity.

- Database connectivity allows an application to communicate with a database management system to store, retrieve, and update information. Two of the most widely used relational databases for this purpose are SQLite3 and MySQL. While both use Structured Query Language (SQL), they feature completely different architectural designs tailored to distinct development needs.

  1 - SQLite3 Connectivity

  SQLite3 runs within the same process as your application, making it lightweight and fast for local storage. It connects directly to a file on your hard drive, meaning no network overhead or external credentials are required.

  Standard Library Integration: Python includes the Python sqlite3 module by default, eliminating the need for separate installation packages.

  Automatic Creation: If the specified database file does not exist during a connection attempt, SQLite3 automatically creates it in the workspace directory.

  In-Memory Alternative: Developers can initiate temporary databases directly in RAM by targeting :memory: as the connection path.

  2 - MySQL Connectivity

  MySQL relies on a client-server architecture, meaning your application acts as a client that must send requests over a network socket to a running MySQL server daemon.

  Driver Dependencies: Connectivity requires an external driver library, such as mysql-connector-python or PyMySQL, to translate commands into network packets.

  Authentication Overhead: Connections require detailed parameters, including host addresses, ports, strict usernames, passwords, and target database schemas.

Connection Pooling: Because establishing network handshakes for every query creates significant system latency, production deployments use connection pools to reuse active pathways.

• Creating and executing SQL queries from Python using these connectors.

- To create and execute SQL queries from Python using database connectors, you must follow a standardized, 4-step workflow: establish a connection, create a cursor, execute the SQL query, and close the resources. While different relational databases use distinct connector libraries, they almost all adhere to the Python Database API Specification v2.0 (PEP 249), meaning the core code structure remains identical across engines.

  The Core 4-Step Workflow

  No matter which database connector you use, your script will follow this structural pipeline:

  python

  # 1. Connect to the database server

  connection = connector.connect(host="...", user="...", password="...")

  # 2. Open a cursor to manage the execution lifecycle

  cursor = connection.cursor()

  # 3. Run the SQL string and safely commit or fetch data

  cursor.execute("YOUR SQL QUERY HERE")

  # 4. Clean up resources to prevent memory leaks

  cursor.close()

  connection.close()

  1 - SQLite (sqlite3)

  SQLite requires no installation because the driver comes pre-bundled with Python's standard library. It reads and writes directly to a local file.

  2 - MySQL (mysql-connector-python)

  For MySQL, you must first install the driver via your terminal: pip install mysql-connector-python.

  3 - PostgreSQL (psycopg2 or psycopg3)

  PostgreSQL uses the psycopg ecosystem. Install it using pip install psycopg2-binary.

  4 - SQL Server (pyodbc)

To interact with Microsoft SQL Server, install the driver using pip install pyodbc.

(10) Search and Match Functions

• Using re.search() and re.match() functions in Python's re module for pattern matching.

- In Python's built-in re module, re.match() checks for a pattern strictly at the beginning of a string, while re.search() scans the entire string to find the first occurrence of that pattern. Both functions return a Match object if a match is successful, or None if no match is found.

• Difference between search and match.

- The exact difference between "search" and "match" depends on the context (e.g., programming, dating apps, or data querying). Generally, search is the act of looking *anywhere* for a query, while match verifies if an item *exactly aligns* with specific criteria or a set starting point.

  1 - In Programming & Regular Expressions (Regex)

  In string and data analysis, the distinction lies in where the system looks:

  Search: Scans the entire text/string for a target pattern and returns the result wherever it first appears. (e.g., checking if the word "cat" is anywhere in "the black cat").

  Match: Checks for the pattern only at the very beginning of the text/string. (e.g., checking if "cat" starts the sentence "cat is here").

  2 - In Databases & Record Systems

  Search: Used to cast a wide net and pull up all available records that contain a specific keyword or meet broad criteria.

  Match: Used to compare two distinct sets of data (like checking a list of applicants against a master registry) to identify duplicates or exact linkages.

  3 - In Online Platforms (e.g., Match.com)

  Search: Allows you to actively type filters (like age, location, and hobbies) and manually browse the profiles that meet your parameters.

  Match: System-generated pairing; an algorithm automatically suggests profiles designed to be highly compatible with your specific preferences and profile traits.